

I'm doing a university lab called "Lab 4: VibeCoding". The goal is to use an AI assistant as a pair-programming partner to fix broken code and add features to a small Python/Pygame game, and to get it done in 3-4 prompts. I will test the code after each prompt and report back. This whole chat will be submitted as part of my grade, so please keep your answers clear and explain your reasoning.

\

ENVIRONMENT

- OS: [Windows/Mac/Linux]
- Python: [version, must be 3.10+]
- pygame: [version, from `pip show pygame`]
- Run with: `python main.py`
- Controls: Left/Right arrow keys move the basket. R restarts after game over.
- Resolution: 700x500 window at 60 FPS

\

PROJECT STRUCTURE

catch-falling/

main.py

requirements.txt

game/game_engine.py, basket.py, falling_object.py, collision.py,
renderer.py

README.md

\

THE GAME

A basket moves along the bottom of the screen. Orange circles spawn at the top at a random x and fall straight down. Each caught object adds 1 to the score. Each object that falls past the bottom adds 1 to the miss counter. At 5 misses the game is over.

\

HOW I WANT YOU TO WORK (applies to every message in this chat)

- Only change what I ask for in each message. Do not refactor, rename, or "improve" unrelated code.
- Keep the existing file layout, the object-oriented structure, and the pygame-only dependency. No new libraries.
- Use clear names and put tunable numbers in named constants at the top of the relevant file.
- For every file you change, give me the COMPLETE updated file contents so I can copy-paste it, and mark each changed or added line with a `# CHANGED` comment so I can review the diff.
- After the code, give me: (1) a short explanation of what was wrong and why your fix works, and (2) a 3-5 step manual test checklist I can follow to verify it.
- If something is ambiguous, state your assumption in one line and proceed instead of asking me questions.
- Do NOT implement anything from later tasks yet. Specifically, leave the unused boost fields in basket.py alone for now.

\

TASKS FOR THIS MESSAGE (Tasks 1 and 2 from the README)

\

Task 1: Fix the falling-object collision bug. The README says: `is\_caught` in game/collision.py only checks whether the object's x is within the basket's left/right edges and never checks height. So an object can score the instant it spawns at the top of the screen as soon as the basket is under its x position. Separately, the catch loop in `update()` in

game/game_engine.py removes objects from the same list it is iterating over, a classic Python mistake that silently skips checking an object when two catchable objects are adjacent in the list. Fix both: a catch must require the object to actually be at basket height, and the catch loop must never skip entries (both of two nearby catchable objects must be credited). Also make sure an object that has already fallen past the basket cannot be "caught" afterwards.

\

Task 2: Improve basket control. The boundary clamp in `handle_input()` ignores the basket's own width, so the basket can sit half off either edge of the screen. Note that `basket.x` is the basket's CENTER. Fix it so the basket always stays fully on screen, and make sure movement stays smooth and responsive while a direction key is held continuously.

\

Here is the full README, followed by every source file.

\

README.md:

Catch the Falling Objects Lab

This project is a single-topic Catch game using **Pygame**. It introduces students to collision timing, a classic list-mutation pitfall, boundary handling, spawn pacing, and temporary power-ups, using a small, readable object-oriented codebase.

What's Provided

A working Catch game with:

- A basket the player moves left and right along the bottom of the screen
- Objects that spawn at the top at random x positions and fall straight down at a fixed interval
- A running score, a miss counter, and a game-over state once too many objects are missed

It has ****one deliberate bug**** (with two distinct symptoms) and ****three features**** left for you to build. You are expected to ****analyze****, ****interact with an AI assistant****, and ****complete/fix**** the game to make it fully functional and more interesting.

**Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.**

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
```bash
pip install -r requirements.txt
```
```

3. Run the game:

```
```bash
python main.py
```
```

****Controls:**** Left/Right arrows to move the basket.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the falling-object collision bug

- > The basket is supposed to catch an object only once it has actually
- > fallen down to basket height. In the current build, ``is_caught`` (in
- > ``game/collision.py``) only checks whether the object's x position is
- > within the basket's left/right edges - it never checks height at
- > all. That means an object can score the instant it spawns at the top
- > of the screen, as soon as the basket happens to be under its x
- > position, however far away it still is. Separately, the loop in
- > ``update()`` (in ``game/game_engine.py``) that checks for catches removes

- > objects from the same list it's iterating over - a classic Python
- > mistake that can silently skip checking an object on a given frame
- > when two catchable objects are next to each other in the list. Fix
- > both: catches should require the object to actually be at basket
- > height, and the catch-checking loop should not skip entries.

Task 2: Improve basket control

- > The basket can currently be moved half off either edge of the
- > screen, since its boundary check doesn't account for its own width.
- > Fix the boundary handling so the basket always stays fully on
- > screen, and make sure movement feels smooth and responsive even
- when
- > a direction key is held down continuously.

Task 3: Implement controlled object spawning

- > Improve the spawning system so objects appear at varied horizontal
- > positions and at varied intervals, rather than a fixed interval with
- > a plain random x each time. Make sure objects never spawn outside
- > the playable area, avoid repeatedly spawning in the same spot, and
- > cap how many objects can be on screen at once so the game never
- > becomes unplayable.

Task 4: Add a temporary basket speed boost

- > Add a speed-boost mechanic the player can activate during play that
- > temporarily increases the basket's movement speed. Show a clear
- > indication while the boost is active, and have the basket
- > automatically return to normal speed once it expires.

Expected Behavior

- An object should only be caught once it has genuinely reached the basket - not the moment its x position happens to line up with the basket, however high up it still is.
- If two catchable objects are near each other, both should be credited - neither should be silently skipped.
- Holding Left or Right continuously should never let the basket move off either edge of the screen.
- Objects should spawn at varied positions and timing, never overwhelming the screen or repeating the same spot constantly.

- The speed boost should be clearly visible while active and wear off on its own.

Folder Structure

...

```
catch-falling/  
├── main.py  
├── requirements.txt  
├── game/  
│   ├── game_engine.py  
│   ├── basket.py  
│   ├── falling_object.py  
│   ├── collision.py  
│   └── renderer.py  
└── README.md
```

...

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay ****before**** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay ****after**** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

\

requirements.txt:

pygame>=2.5.0

\

\

main.py:

```
"""
```

Catch the Falling Objects (Lab Starter)

```
\
```

Run with: `python3 main.py`

```
\
```

Controls: Left/Right arrows to move the basket.

```
"""
```

```
\
```

```
import pygame
```

```
\
```

```
from game.game_engine import GameEngine
```

```
from game.renderer import WINDOW_SIZE
```

```
\
```

```
\
```

```
def main():
```

```
    pygame.init()
```

```
    screen = pygame.display.set_mode(WINDOW_SIZE)
```

```
    pygame.display.set_caption("Catch the Falling Objects")
```

```
    clock = pygame.time.Clock()
```

```
    font = pygame.font.SysFont("consolas", 22)
```

```
\
```

```
    engine = GameEngine()
```

```
running = True
```

```
while running:
```

```
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
```

```
            running = False
```

```
        elif event.type == pygame.KEYDOWN:
```

```
            engine.handle_keydown(event.key)
```

```
    \
```

```
    keys = pygame.key.get_pressed()
```

```
    engine.handle_input(keys)
```

```
    engine.update()
```

```
    engine.draw(screen, font)
```

```
    \
```

```
    pygame.display.flip()
```

```
    clock.tick(60)
```

```
    \
```

```
    pygame.quit()
```

```
    \
```

```
    \
```

```
if __name__ == "__main__":
```

```
    main()
```

```
game/game_engine.py:
```

```
"""
```

GameEngine: owns the basket and all falling objects.

Starter version: basket movement and spawning both work at a basic level (Tasks 2 and 3 ask you to improve them), there's no speed boost yet (Task 4 builds it from scratch), and catch detection has two known bugs (see game/collision.py and the catch-checking loop below) that Task 1 asks you to fix.

```
"""
```

```
import random
```

```
import pygame
```

```
from game.basket import Basket
```

```
from game.falling_object import FallingObject
```

```
from game.collision import is_caught
```

```
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 50
```

```
MAX_MISSES = 5
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
```

```
        self.objects = []
```

```
        self.frames_until_spawn = 0
```

```
        self.score = 0
```

```
        self.misses = 0
```

```
        self.game_over = False
```

```
    def _spawn_object(self):
```

```
        x = random.randint(20, WIDTH - 20)
```

```
        self.objects.append(FallingObject(x=x, y=-14, speed=3))
```

```
    def handle_input(self, keys_pressed):
```

```
        if self.game_over:
```

```
            return
```

```
        if keys_pressed[pygame.K_LEFT]:
```

```
            self.basket.x -= self.basket.speed
```

```
        if keys_pressed[pygame.K_RIGHT]:
```

```
            self.basket.x += self.basket.speed
```

```
        # Boundary handling: only clamps against the screen edges, not
```

```
        # accounting for the basket's own width - it can hang half off
```

```

# either side of the screen.
self.basket.x = max(0, min(WIDTH, self.basket.x))

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self.__init__()

def update(self):
    if self.game_over:
        return

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        self._spawn_object()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for obj in self.objects:
        obj.update()

    basket_rect = self.basket.get_rect()
    for obj in self.objects:          # BUG: mutating this list while
iterating over it
        if is_caught(basket_rect, obj):
            self.score += 1
            self.objects.remove(obj)

    missed = [o for o in self.objects if o.is_past_bottom(HEIGHT)]
    if missed:
        self.objects = [o for o in self.objects if not
o.is_past_bottom(HEIGHT)]
        self.misses += len(missed)
        if self.misses >= MAX_MISSES:
            self.game_over = True

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.basket, self.objects)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Misses:
{self.misses}/{MAX_MISSES}", (10, 36))

    if self.game_over:
        renderer.draw_banner(surface, font, f"Game Over! Final score:
{self.score}. Press R to restart.")

```

```
\
```

game/basket.py:

```
"""
```

Basket: the player-controlled catcher at the bottom of the screen.

```
"""
```

```
\
```

```
import pygame
```

```
\
```

```
\
```

```
class Basket:
```

```
    def __init__(self, x, y, width=90, height=24, speed=5):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.width = width
```

```
        self.height = height
```

```
        self.speed = speed
```

```
        self.boosted_frames = 0
```

```
        self.normal_speed = speed
```

```
        self.boost_speed = speed
```

```
\
```

```
    def get_rect(self):
```

```
        return pygame.Rect(
```

```

        int(self.x - self.width / 2), int(self.y - self.height / 2),

        self.width, self.height,

    )
\
\

```

game/falling_object.py:

```

"""

```

FallingObject: a simple object that falls straight down.

```

"""

```

```

\

```

```

\

```

```

class FallingObject:

```

```

    def __init__(self, x, y, radius=14, speed=3, color=(230, 140, 60)):

```

```

        self.x = x

```

```

        self.y = y

```

```

        self.radius = radius

```

```

        self.speed = speed

```

```

        self.color = color

```

```

\

```

```

    def update(self):

```

```

        self.y += self.speed

```

```

\

```

```
def is_past_bottom(self, height):  
  
    return self.y - self.radius > height
```

game/collision.py:

```
"""
```

collision: figures out whether a falling object is within the basket.

```
"""
```

```
\
```

```
\
```

```
def is_caught(basket_rect, obj):
```

```
    return basket_rect.left <= obj.x <= basket_rect.right
```

```
\
```

```
\
```

game/renderer.py:

```
"""
```

renderer: all pygame drawing lives here, kept separate from game logic.

```
"""
```

```
import pygame
```

```
WIDTH, HEIGHT = 700, 500
```

```
WINDOW_SIZE = (WIDTH, HEIGHT)
```

```
COLOR_BG = (25, 30, 45)
```

```
COLOR_BASKET = (150, 110, 70)
```

```
COLOR_TEXT = (255, 255, 255)
```

```
def draw_scene(surface, basket, objects):
```

```
    surface.fill(COLOR_BG)
```

```
    for obj in objects:
```

```
        pygame.draw.circle(surface, obj.color, (int(obj.x), int(obj.y)),
```

```

obj.radius)
    pygame.draw.rect(surface, COLOR_BASKET, basket.get_rect(),
border_radius=6)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2,
surface.get_height() // 2))
    surface.blit(surf, rect)

```

Tasks 1–2

Assumption: “At basket height” means the falling circle must overlap the basket’s rectangular area; an object below the basket is not catchable.

Only **two files need to change**: `game/collision.py` and `game/game_engine.py`. `basket.py` is intentionally left unchanged, including its unused boost fields.

1. `game/collision.py`

 Python

```

"""
collision: figures out whether a falling object is within the basket.
"""

def is_caught(basket_rect, obj):
    # CHANGED: Check both horizontal and vertical overlap so objects are
    # CHANGED: only caught when they have actually reached the basket.
    obj_left = obj.x - obj.radius # CHANGED
    obj_right = obj.x + obj.radius # CHANGED
    obj_top = obj.y - obj.radius # CHANGED
    obj_bottom = obj.y + obj.radius # CHANGED

    # CHANGED: Require the falling object's circle bounds to overlap
    # CHANGED: the basket bounds in both directions.
    horizontal_overlap = obj_right >= basket_rect.left and obj_left <= basket_rect.right # CHANGED
    vertical_overlap = obj_bottom >= basket_rect.top and obj_top <= basket_rect.bottom # CHANGED

    return horizontal_overlap and vertical_overlap # CHANGED

```

2. `game/game_engine.py`

</> Python

"""

GameEngine: owns the basket and all falling objects.

Starter version: basket movement and spawning both work at a basic level (Tasks 2 and 3 ask you to improve them), there's no speed boost yet (Task 4 builds it from scratch), and catch detection has two known bugs (see game/collision.py and the catch-checking loop below) that Task 1 asks you to fix.

"""

```
import random
import pygame
```

```
from game.basket import Basket
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 50
MAX_MISSES = 5
```

What was wrong and why this fixes it

Task 1 — Collision

There were two separate bugs:

1. Collision only checked X position.

An object was considered caught whenever its X coordinate was between the basket's edges, regardless of how high it was.

The new `is_caught()` checks both:

- horizontal overlap
- vertical overlap

Therefore, an object near the top of the screen cannot be caught merely because it is horizontally aligned with the basket.

2. The catch loop modified `self.objects` while iterating over it.

For example, if objects `A` and `B` were both catchable:

</> Plain text

```
[A, B]
```

```
↑
```

```
remove A
```

Python's iteration then advances to the next index, potentially skipping `B`.

The new code creates `remaining_objects` instead. Every object is examined exactly once, so two adjacent catchable objects both increase the score.

An object that has fallen below the basket cannot satisfy the vertical-overlap check, so it cannot be caught after passing the basket.

Task 2 — Basket movement

`basket.x` is the **center** of the basket, but the old code clamped that center between `0` and `WIDTH`.

That allowed:

</> Plain text

```
| basket
|
0
```

with half the basket outside the screen.

The new boundaries are:

</> Plain text

```
minimum x = basket.width / 2
maximum x = WIDTH - basket.width / 2
```

so the entire basket remains visible.

Movement is still performed once every game frame while the key is held, so the existing continuous-key behavior remains smooth and responsive. No acceleration or unrelated movement system was introduced.

Manual test checklist

1. **Run the original game:** `python main.py` and move the basket under a newly spawned object. Verify the score does **not** increase while the object is still high on the screen.
2. **Test a normal catch:** Keep the basket underneath an object until it reaches the basket. Verify the score increases by exactly 1.
3. **Test two nearby objects:** Arrange for two objects to reach the basket together. Verify both are caught and the score increases by 2 rather than 1.
4. **Test basket boundaries:** Hold **Left** continuously until the basket reaches the edge, then hold **Right** continuously until it reaches the other edge. Verify no part of the basket leaves the window.
5. **Test misses:** Let objects fall without catching them. Verify they increase the miss counter and that an object falling below the basket does not subsequently increase the score.

Do not change `basket.py` yet. Tasks 3 and 4 will build on the remaining starter code later.

Same working rules as before: only change what I ask, complete updated files with `# CHANGED` markers, a short explanation, and a manual test

checklist. Do not touch the speed boost yet.

TASK 3: Implement controlled object spawning (README Task 3). The README says: "Improve the spawning system so objects appear at varied horizontal positions and at varied intervals, rather than a fixed interval with a plain random x each time. Make sure objects never spawn outside the playable area, avoid repeatedly spawning in the same spot, and cap how many objects can be on screen at once so the game never becomes unplayable."

Current behavior: `_spawn_object()` in `game/game_engine.py` picks `random.randint(20, WIDTH - 20)` and `SPAWN_INTERVAL_FRAMES = 50` is a fixed interval.

Specific requirements:

1. Varied intervals: after each spawn, pick the next interval randomly between a named minimum and maximum number of frames (suggest roughly 30 to 80 frames at 60 FPS).
2. Varied x positions that are always fully inside the playable area, accounting for the object's radius so no circle is clipped by the left or right edge.
3. Avoid repeats: remember the previous spawn x and enforce a named minimum horizontal distance from it (suggest around 60-100 px). Make sure this cannot loop forever or fail on a 700px-wide screen, and explain how you guarantee that.
4. Cap: define a named `MAX_OBJECTS_ON_SCREEN` (suggest 6). When the cap is reached, skip spawning this time. Explain how the spawn timer behaves in that case.
5. Reset the spawn-related state (previous x, timer) correctly when the game restarts with R.
6. Keep the code small and readable.

Then give me the test checklist, including how I can visually confirm the variation, the spacing, and the cap.

Task 3 — Controlled Object Spawning

Assumption: The spawn-spacing rule applies between consecutive actual spawns. If spawning is skipped because the screen is at capacity, the previous spawn position remains unchanged.

Only `game/game_engine.py` needs to change. `basket.py`, `collision.py`, and all other files remain untouched.

`game/game_engine.py`

 Python

```
"""
```

```
GameEngine: owns the basket and all falling objects.
```

```
Starter version: basket movement and spawning both work at a basic
level (Tasks 2 and 3 ask you to improve them), there's no speed boost
yet (Task 4 builds it from scratch), and catch detection has two
known bugs (see game/collision.py and the catch-checking loop below)
that Task 1 asks you to fix.
```

```
"""
```

```
import random
```

```
import pygame
```

```
from game.basket import Basket
```

```
from game.falling_object import FallingObject
```

```
from game.collision import is_caught
```

```
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_MIN_INTERVAL_FRAMES = 30 # CHANGED
```

```
SPAWN_MAX_INTERVAL_FRAMES = 80 # CHANGED
```

```
MIN_SPAWN_DISTANCE = 80 # CHANGED
```

```
MAX_OBJECTS_ON_SCREEN = 6 # CHANGED
```

What changed and why

1. Varied spawn intervals

The old fixed:

 Python

```
SPAWN_INTERVAL_FRAMES = 50
```

has been replaced with:

 Python

```
SPAWN_MIN_INTERVAL_FRAMES = 30
```

```
SPAWN_MAX_INTERVAL_FRAMES = 80
```

After every spawn **or skipped spawn attempt**, the next interval is randomly selected between 30 and 80 frames.

At 60 FPS, that corresponds to approximately **0.5–1.33 seconds** between spawn attempts.

2. Objects stay fully inside the screen

The object's radius is 14 pixels, so its center must stay between:

 Plain text



Therefore:

`</>` Python

```
SPAWN_X_MIN = OBJECT_RADIUS
SPAWN_X_MAX = WIDTH - OBJECT_RADIUS
```

The circle cannot be clipped horizontally.

3. Minimum distance between spawn positions

`MIN_SPAWN_DISTANCE` is set to **80 pixels**.

The code makes up to 20 random attempts to find a suitable position. If those attempts happen to fail, it **does not loop indefinitely**.

It instead compares the two extreme valid positions:

`</>` Plain text



and chooses whichever is farther from the previous spawn.

This guarantees a valid position because the playable center range is **672 pixels wide**, while the required separation is only **80 pixels**. In fact, at least one endpoint will always be at least 336 pixels away from any previous position in the playable range.

4. Maximum of six active objects

`</>` Python

```
MAX_OBJECTS_ON_SCREEN = 6
```

When six objects are already active, the game **does not create another object**.

The timer is still reset to another random 30–80 frame interval. This prevents the game from repeatedly attempting to spawn every frame while the screen is full.

Once an object is caught or missed and the count drops below six, a later scheduled spawn can occur normally.

5. Restart behavior

`previous_spawn_x` is initialized in `__init__()` :

`</>` Python

```
self.previous_spawn_x = None
```

Since `handle_keydown()` already calls:

`</>` Python

```
self.__init__()
```

when `R` is pressed after game over, the spawn position history and spawn timer are both reset automatically.

The first object after a restart can therefore spawn anywhere in the valid horizontal range.

Manual test checklist

1. Check interval variation

- Run `python main.py` .
- Watch the first several objects spawn.
- They should clearly not appear at a perfectly fixed rhythm.
- At 60 FPS, the approximate interval should vary between **0.5 and 1.33 seconds**.

2. Check horizontal boundaries

- Watch objects spawn near both sides of the window.
- Verify that the orange circles remain completely visible and no circle is clipped by the left or right edge.

3. Check spawn spacing

- Watch consecutive objects as they appear.
- Their spawn positions should not repeatedly be close together.
- Consecutive spawn centers should be at least approximately **80 pixels apart**.

4. Check the six-object cap

- Avoid catching objects and let several accumulate.
- Count the active objects.
- Once there are 6, no seventh object should appear until one of the existing objects is caught or missed.

5. Check restart reset

- Let the game reach Game Over.
- Press `R` .
- Verify that the score/misses and object list reset and the first new object is not constrained by the previous game's final spawn position.

Same working rules: only change what I ask, complete updated files with **# CHANGED** markers, a short explanation, and a manual test checklist.

TASK 4: Add a temporary basket speed boost (README Task 4). The README says: "Add a speed-boost mechanic the player can activate

during play that temporarily increases the basket's movement speed. Show a clear indication while the boost is active, and have the basket automatically return to normal speed once it expires."

Note that `game/basket.py` already has unused fields `boosted_frames`, `normal_speed`, and `boost_speed`. Please use and tidy those up instead of adding duplicate state.

My design decisions (follow these):

- Activation key: SPACE, handled as a single key press (via `handle_keydown`), not as a held key.
- Basket normal speed stays 5; boost speed is 9 pixels per frame (named constants).
- Boost duration: 180 frames (3 seconds at 60 FPS). Use frame counters, consistent with the rest of the game.
- Cooldown: 300 frames (5 seconds) after activation before the boost can be used again, so it can't be spammed. Pressing SPACE during an active boost or a cooldown does nothing.
- Pressing SPACE after game over does nothing.
- On restart with R, boost and cooldown state must fully reset.

Display requirements (drawing code belongs in `game/renderer.py`, logic in the `basket/engine`):

- While the boost is active: a clear on-screen label such as "BOOST ACTIVE" with the remaining seconds, AND the basket changes color so the effect is visible on the basket itself.
- When the boost is available: a small "Boost ready (SPACE)" hint. During cooldown: show a cooldown indicator.
- The basket returns to its normal color and speed automatically when the boost ends.

Correctness requirements:

- The higher speed must still respect the boundary clamp from Task 2, so the basket never leaves the screen even when boosted.
- Collision detection must still work correctly at the higher speed.

Then give me the test checklist.

Task 4 — Temporary Basket Speed Boost

I will change only the three files required for this feature:

- `game/basket.py` — boost state and speed handling
- `game/game_engine.py` — SPACE activation and frame timing

- `game/renderer.py` — `boost/ready/cooldown` display and basket color

The Task 2 boundary clamp and Task 3 spawning behavior remain unchanged.

1. `game/basket.py`

`</>` Python

"""

Basket: the player-controlled catcher at the bottom of the screen.

"""

import pygame

NORMAL_SPEED = 5 # CHANGED

BOOST_SPEED = 9 # CHANGED

BOOST_DURATION_FRAMES = 180 # CHANGED

BOOST_COOLDOWN_FRAMES = 300 # CHANGED

class Basket:

def __init__(self, x, y, width=90, height=24, speed=NORMAL_SPEED): # CHANGED

self.x = x

self.y = y

self.width = width

self.height = height

self.normal_speed = speed # CHANGED

self.boost_speed = BOOST_SPEED # CHANGED

self.speed = self.normal_speed # CHANGED

"""

2. `game/game_engine.py`

`</>` Python

"""

GameEngine: owns the basket and all falling objects.

Starter version: basket movement and spawning both work at a basic level (Tasks 2 and 3 ask you to improve them), there's no speed boost yet (Task 4 builds it from scratch), and catch detection has two known bugs (see `game/collision.py` and the catch-checking loop below) that Task 1 asks you to fix.

"""

import random

import pygame

from game.basket import Basket

from game.falling_object import FallingObject

from game.collision import is_caught

from game.renderer import WIDTH, HEIGHT

SPAWN_MIN_INTERVAL_FRAMES = 30

```
SPAWN_MAX_INTERVAL_FRAMES = 80
MIN_SPAWN_DISTANCE = 80
MAX_OBJECTS_ON_SCREEN = 6
```

3. game/renderer.py

`</>` Python

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (25, 30, 45)
COLOR_BASKET = (150, 110, 70)
COLOR_BOOST_BASKET = (70, 180, 255) # CHANGED
COLOR_TEXT = (255, 255, 255)
COLOR_BOOST_TEXT = (80, 220, 255) # CHANGED
COLOR_COOLDOWN_TEXT = (255, 190, 80) # CHANGED

def draw_scene(surface, basket, objects):
    surface.fill(COLOR_BG)
    for obj in objects:
        pygame.draw.circle(
            surface,
```

What was wrong / why this works

The basket already contained three unused boost-related fields, so I extended those rather than creating duplicate state.

Activation

Pressing **SPACE** calls `activate_boost()` once through `KEYDOWN`.

When available:

- normal speed = **5 px/frame**
- boost speed = **9 px/frame**
- boost duration = **180 frames = 3 seconds**
- cooldown = **300 frames = 5 seconds**

SPACE does nothing while either `boosted_frames` or `cooldown_frames` is nonzero.

The cooldown starts at activation, so the boost becomes available again **300 frames after activation**, rather than 300 frames after the boost finishes.

Automatic expiration

`update_boost()` decrements the counters once per game frame. When `boosted_frames` reaches zero:

`</>` Plain text

```
speed: 9 → 5
```

The basket therefore automatically returns to its normal speed.

The counters are updated **after movement and collision processing for the current frame**, preventing the activation frame from immediately losing one full frame of boost duration.

Boundary handling

The Task 2 clamp remains in place and uses the basket's width:

`</>` Plain text

```
minimum center = basket.width / 2
maximum center = WIDTH - basket.width / 2
```

Since boosted movement still passes through this same clamp, the basket cannot leave the screen even at 9 px/frame.

Visual indication

While active:

- basket becomes blue
- `BOOST ACTIVE - Ns` appears on screen

When available:

- `Boost ready (SPACE)` appears

During cooldown:

- `Boost cooldown - Ns` appears

The renderer only handles the visual state; the basket and engine maintain the boost logic.

Restart

There is one important correction to the `handle_keydown()` structure: because the method immediately returns when `self.game_over` is true, `R` must be handled **before** that return. The correct implementation is therefore this version:

`</>` Python

```
def handle_keydown(self, key):
    if key == pygame.K_r and self.game_over: # CHANGED
        self.__init__() # CHANGED
        return # CHANGED

    if self.game_over:
```

```
return
```

```
if key == pygame.K_SPACE:  
    self.basket.activate_boost()
```

Use that corrected `handle_keydown()` in the `game_engine.py` file above. This preserves the existing restart behavior while ensuring SPACE remains disabled after game over.

On restart, `self.__init__()` creates a new `Basket`, which resets:

- `boosted_frames`
- `cooldown_frames`
- `speed`

as well as the existing score, misses, objects, spawn timer, and previous spawn position.

Manual test checklist

1. Test activation and speed

- Start the game.
- Confirm `Boost ready (SPACE)` is visible.
- Press **SPACE once**.
- Confirm the basket changes color and `BOOST ACTIVE - 3s` appears.
- Hold Left/Right and visually confirm the basket moves noticeably faster than its normal speed.

2. Test duration and automatic reset

- Activate the boost.
- Watch the countdown decrease from approximately 3 seconds.
- After roughly 3 seconds, confirm the basket returns to its normal color and movement speed.
- Confirm the display changes to the cooldown indicator.

3. Test cooldown

- Press SPACE repeatedly while the cooldown indicator is visible.
- Confirm another boost does **not** activate.
- After approximately 5 seconds from the original activation, confirm `Boost ready (SPACE)` returns.
- Press SPACE once and verify the boost activates again.

4. Test boundaries and collision

- Activate the boost and hold Left until the basket reaches the left edge.
- Hold Right until it reaches the right edge.
- Confirm the entire basket remains on screen.
- Catch objects while boosted and verify the score still increases normally.

5. Test game over and restart

- Allow the game to reach Game Over.
- Press SPACE and confirm nothing happens.
- Press **R**.
- Confirm the game restarts with normal basket speed, no active boost, no cooldown, and the `Boost ready (SPACE)` indicator.